

Range Query Data Structure

Segment Trees

↳ min / max Range Query

↳ point updates

↳ sum Range Query.

↳ Lazy Propagation in segment trees.

• Fenwick Trees | Binary Indexed Trees

What is the use case when is it needed??

ex: arr = $\{1, 0, 3, 2, 4, 2, 1\}$ $n=7$.

3 queries are given.

$Q = [L, R]$ get the min element in the range.

$[1, 3] \Rightarrow \min = 0$.

$[3, 6] \Rightarrow \min = 1$

$[2, 5] \Rightarrow \min = 2$.

Q. min in Range $[L, R] \Rightarrow$ given Q queries.

$1 \leq \text{queries.size} \leq 10^5$ and Array size $\leq 10^5$.

↳ For min, Max Queries cannot use Prefix Sum logic.

$\Rightarrow$ For this Above Question. $\Rightarrow$ must use Segment Trees.

Brute Force = $O(Q \times (R-L+1)) \approx O(Q \times N)$
 $\approx O(N^2) \Rightarrow \text{TLE}$

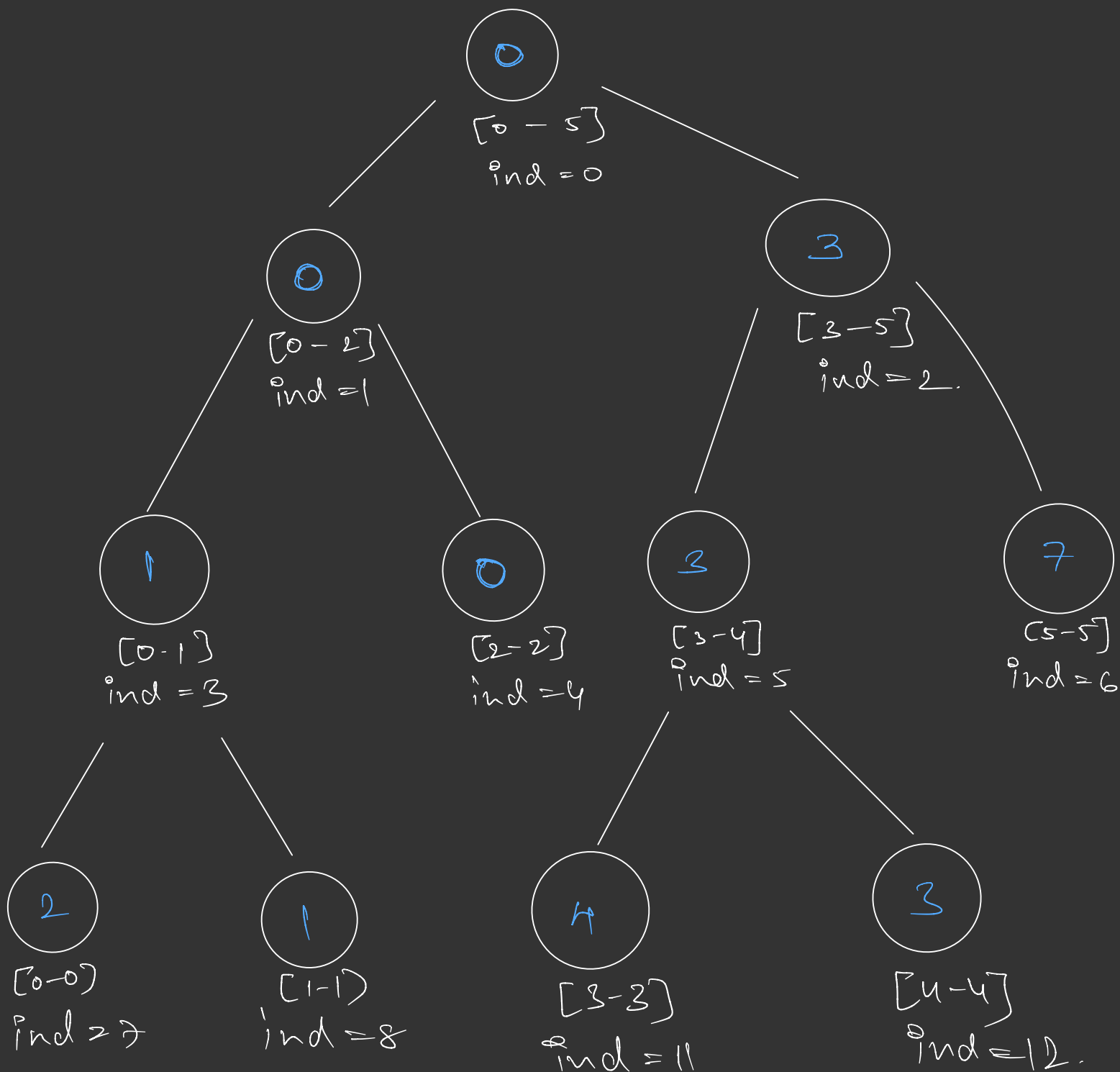
↳ This is where Segment Trees come into the picture.

↳ answers the queries in $O(\log(N))$

Eg of segment tree $\Rightarrow$ 0 based indexing.

arr[] = { 2, 1, 0, 4, 3, 7 }

0 1 2 3 4 5



This is how a segment tree looks.

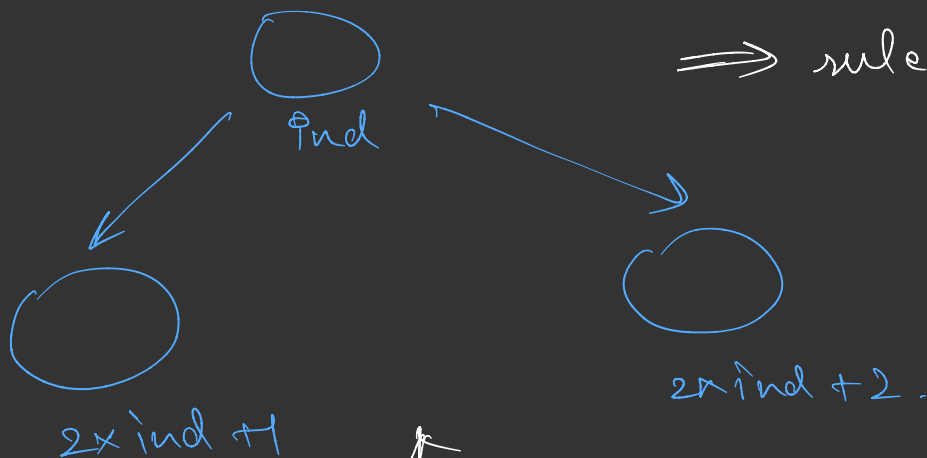
So get the max/min by recursively breaking down.
 $f() \rightarrow f() \rightarrow$ update the min/max.
(like postorder)

Note: Leaf nodes of the segment Tree are the elements of the Array itself

0	0	3	1	0	3	7	2	1				4	3
0	1	2	3	4	5	6	7	8	9	10	11	12	

↳ segment tree.

Simple rule. $\Rightarrow$ Rule for indexing of nodes.



↳ in 0 based indexing.

Left child = $2 * ind + 1$.

Right child = $2 * ind + 2$.

low - high $\Rightarrow$ we start with $f(ind, low, high)$

build(ind, low, high) {

if (low == high) { $\Rightarrow$ Base Case single element

tree[ind] = arr[low]; return;

}

mid = (low + high) / 2

f(2 * ind + 1, low, mid)

f(2 * ind + 2, mid + 1, high)

left and right will be computed.

tree[ind] = min(tree[2 * ind + 1], tree[2 * ind + 2])

}

$\Rightarrow$ This is the code for Build of Segment Tree

Segment tree size $\Rightarrow$ vector<int> tree.

$\hookrightarrow$ if $O(4n)$ is taken $\Rightarrow$ it is sufficient.

size $\rightarrow O(4n)$

Query $\Rightarrow$ Find the min(L, R).

$\hookrightarrow$ Results in 3 cases of range to check.

① partial overlap with range (L, R)

left node $\rightleftarrows$

right node $\rightleftarrows$

return (left node + right node)

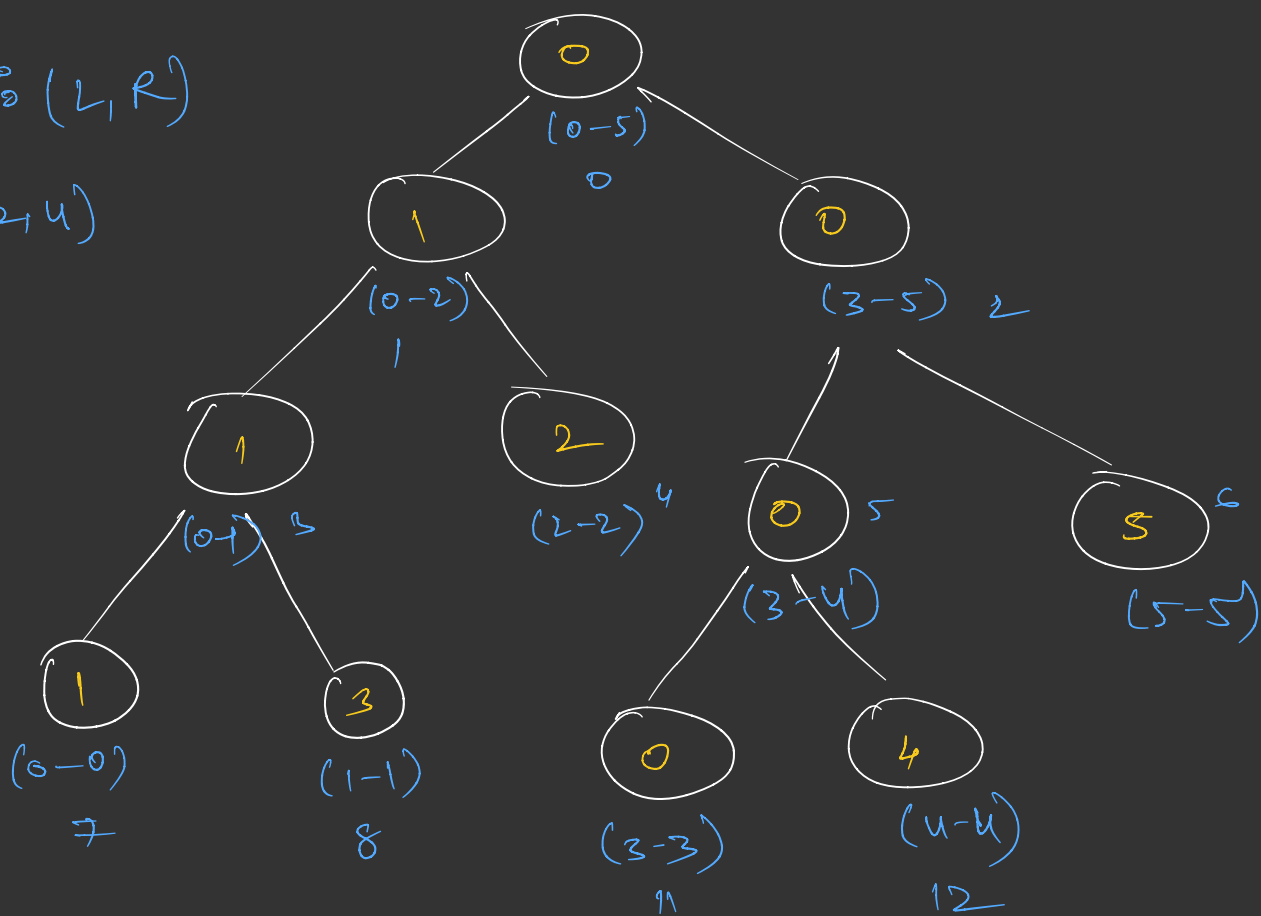
② no overlap with range (L, R)
return INT_MAX | INT_MIN

③ complete overlap with (L, R)
return tree[ind]

understanding the three cases.

arr = { 1, 3, 2, 0, 4, 5 }
 0 1 2 3 4 5

eg: (L, R)
 $(2, 4)$



partial overlap. $\Rightarrow$ any type of overlap.

eg: Range = $(2, 4)$ and Range = $(3, 5)$.

$\Downarrow$
partial overlap. $\Rightarrow$ go left and right.
└─ some portion overlaps.

no. overlap. $\Rightarrow$ Range = $(2, 4)$ and Range = $(5, 6)$.

$\hookrightarrow$ return INT_MAX . $\Rightarrow$ does not contribute to the Ans

complete overlap $\Rightarrow$ Range 1 = (2, 4), Range 2 = (2, 3).

Range 2 completely lies inside.

Note: Given Query Range = (L, R).

Searching Range = (L', R')

(Searching Query Range must lie inside given Query Range for complete overlap.) $\Rightarrow$ then complete overlap.

query(ind, low, high, l, r)

{ // no overlap $\Rightarrow$ [low, high] ... [l, r] or [l, r] ... [low, high]

if (high < l || r < low)

{ return INT_MAX;

}

// complete overlap. $\Rightarrow$ [l ... low, ..., high ... r]

if (l <= low and high <= r)

{ return tree[ind]

}

// case left is partial overlap.

else {

mid = (low + high) / 2

left = query(2*ind+1, low, mid, l, r)

right = query(2*ind+2, mid+1, high, l, r)

return min(left, right)

}

}

Point Updates

arr = [2, 1, 0, 3, 5, 4].

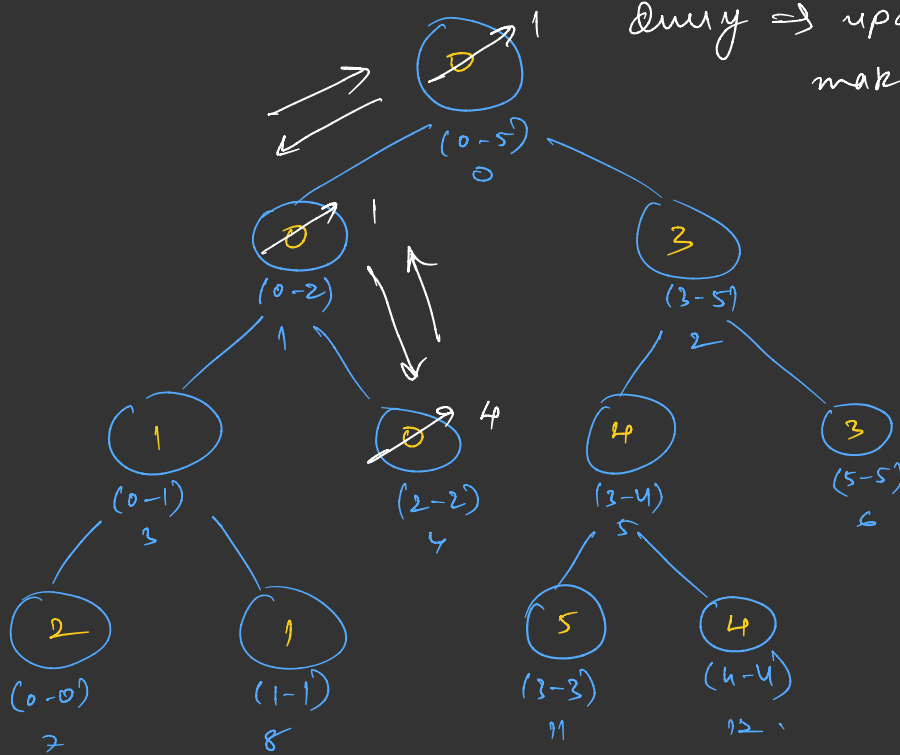
queries $\Rightarrow$ Type 1 Q. and Type 2 Q..

$\uparrow$
give the min(L, R)

$\uparrow$ ind, val.
update $a[ind] = val$.

$\hookrightarrow$ For this segment tree needs to be updated.

eg: arr = [2, 1, 0, 3, 5, 4].



Query $\Rightarrow$ update index = 2
make arr[2] = 4.

update(ind, low, high, i, val)

if (low == high) $\Rightarrow$ low == high == val. (search like BST)

tree[i] = val;
return;

}

mid = (low + high) / 2

if (i <= mid) update(2*ind+1, low, mid, i, val)

else update(2*ind+2, mid+1, high, i, val)

// after updating either in left or right subtree. $\Rightarrow$ update the min

tree[ind] = min(tree[2*ind+1], tree[2*ind+2])

}

Time Complexity

Build $\Rightarrow$ similar to Build max heap or Min heap.

$\hookrightarrow O(N) \Rightarrow$ no. of nodes created.

Query $\Rightarrow O(\log N) \Rightarrow O(\text{Height of the tree})$

$\hookrightarrow$ worst case will be when $[1, \dots, 1] \Rightarrow$ very small range.

update $\Rightarrow O(\log N)$

Question

2 arrays are given. and Queries of Type 1 and Type 2

Type 1 $\Rightarrow$ Min Queries.

$\boxed{1 \quad L1 \quad R1 \quad L2 \quad R2}$ $\Leftarrow$ get overall min.
 $\Rightarrow \min(L1, R1), (L2, R2)$

Type 2 $\Rightarrow$ update Q

2, 1, index, val or 2, 2, index, val.

$\hookrightarrow$ update Array 1

$\hookrightarrow$ update Array 2.

$\Rightarrow$ Range Based Queries for min and point updates use Segment Trees.

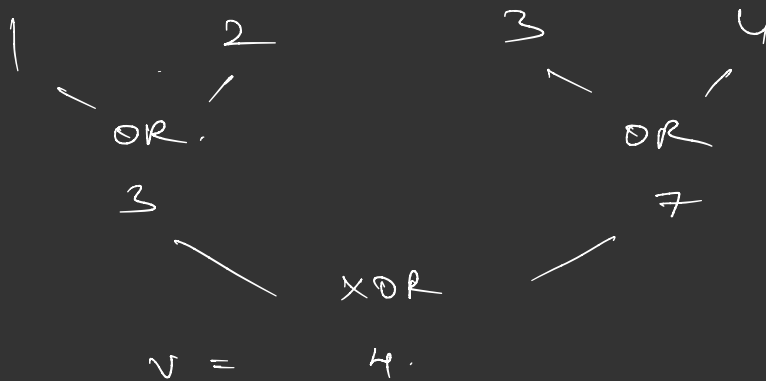
Use 2 segment Trees. $\Rightarrow$ seg1, seg2.

Xenia and Bit Operations

2^n elements in the Array.

iterations to be done $\Rightarrow$ 2 Adj elements OR.
then next iteration take XOR.

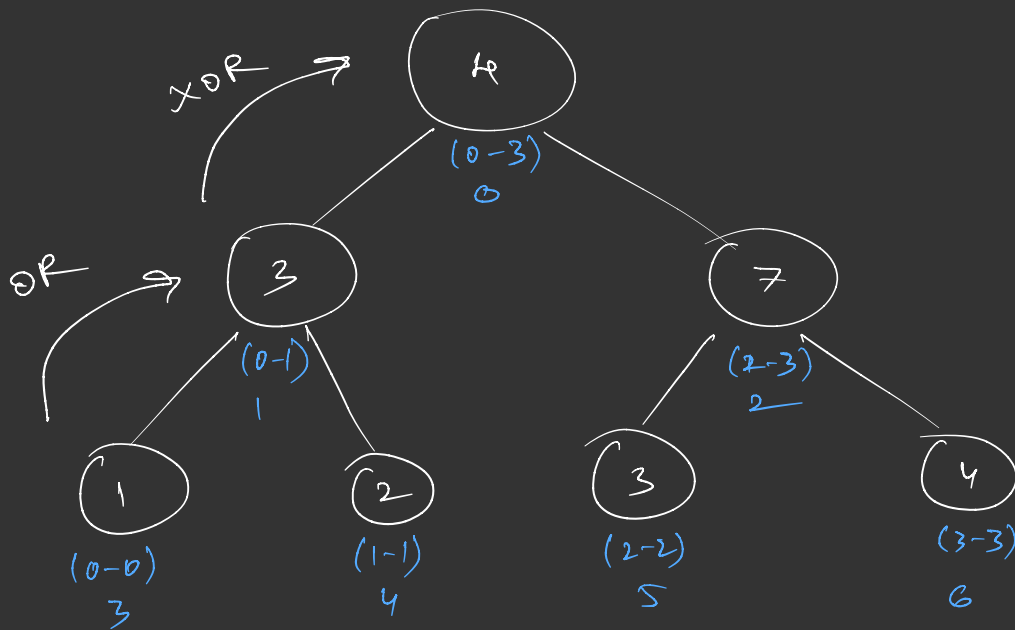
eg: arr = (1, 2, 3, 4)



We are suppose to make n Queries.

Where we update the Result $\Rightarrow a[p] = b$.

Segment Tree.



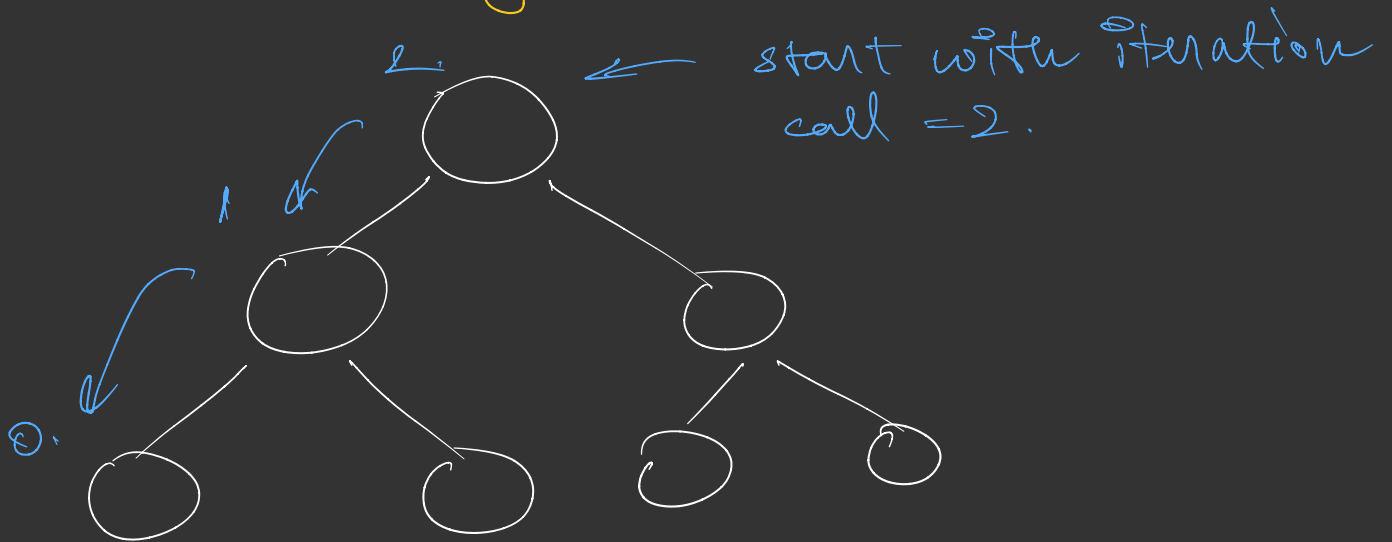
Question is how will we Build the segment trees.
So it is alternate OR and XOR Bitwise operations.

Array has 2^n elements.

so. n iterations will be needed.

Building the segment tree.

we need a tracker for which iteration that we are doing.



when the iteration call = odd. (1, 3, 5...)
do. OR. otherwise even iteration call
apply XOR.

$\text{if (iteration = odd)}$

$\text{tree[ind]} = \text{tree}[2*\text{ind}+1] \mid \text{tree}[2*\text{ind}+2]$

else

$\text{tree[ind]} = \text{tree}[2*\text{ind}+1] \wedge \text{tree}[2*\text{ind}+2]$

In this way the $\text{segtree}[0] = v$.

In this way we can make updates to the tree.

update will use the same logic where we go to the leaf make the changes at the leaf level.

V. imp

Sereja and Brackets

Bracket sequence $\Rightarrow s$ is given.

and q queries are given.

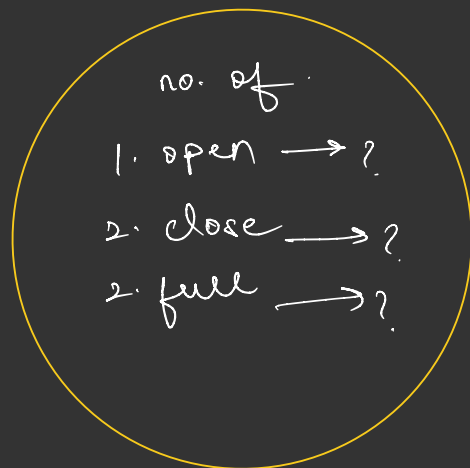
$q[i] = [l_i, r_i] \Rightarrow$ return the max length of the correct bracket sequence.

Eg: $s = " ()) (()) (()) ("$

$\Rightarrow$ involves complicated usage of segment trees.

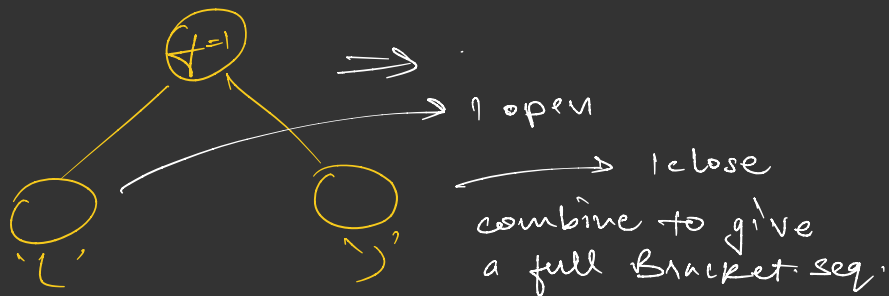
Nodes in segment Tree

$\hookrightarrow$ store multiple information in a node in the segment tree.



(low-high)
ind.

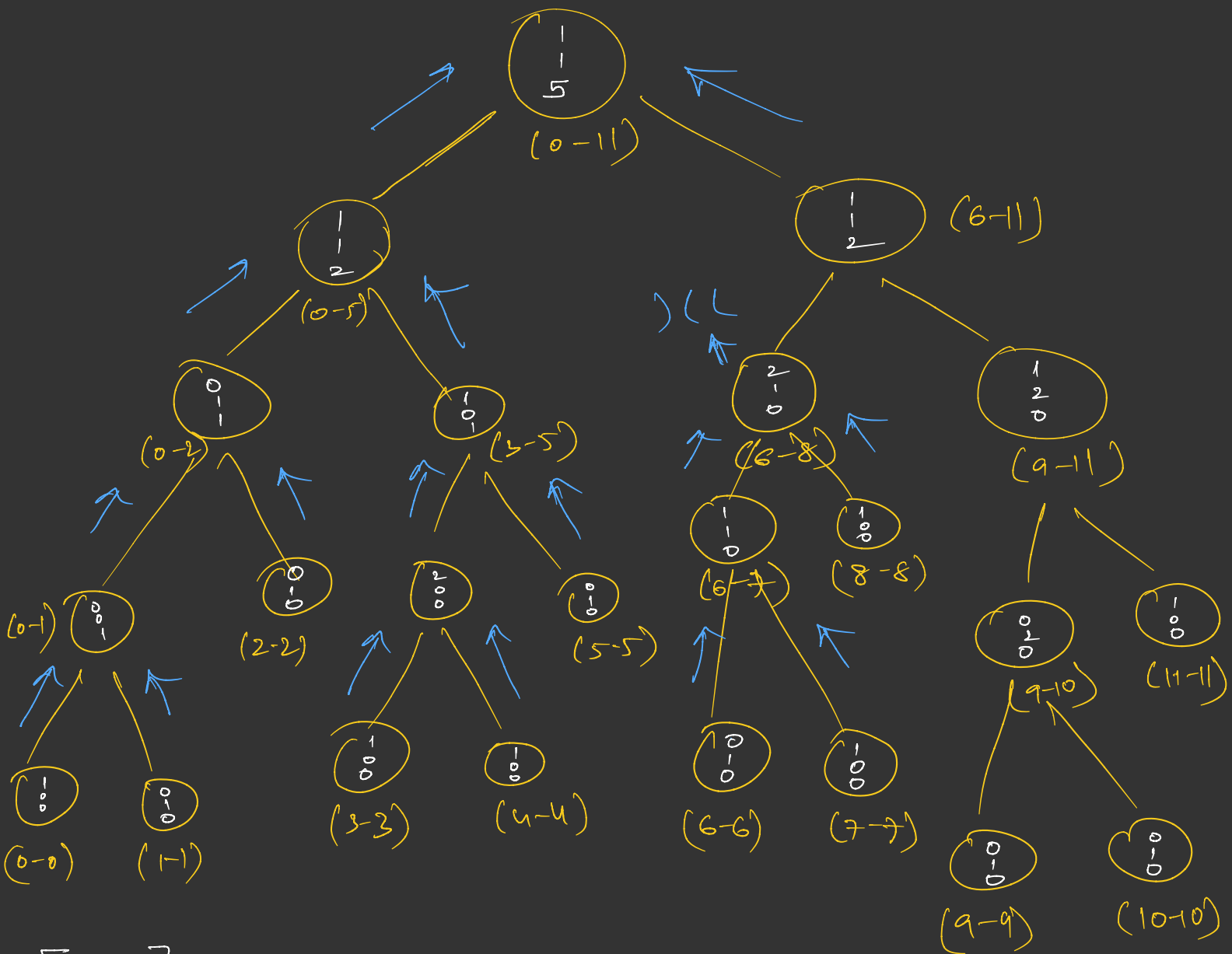
How to determine if there is a full bracket sequence?



$\min \left(\begin{array}{l} \text{opening from} \\ \text{the left} \end{array}, \begin{array}{l} \text{closing from} \\ \text{the right} \end{array} \right) = \text{full Bracket Sequence..}$

* This logic can also be used for solving other parenthesis Questions for longest valid parenthesis.

eg: ()) (()) (()) ()
0 1 2 3 4 5 6 7 8 9 10 11



open
close
full

1 open + 1 close = full
and add rest as carry over.

fulls add up.

open $\Rightarrow$ left and closing $\Rightarrow$ Right.

only these two combine to form
a full Bracket Sequence.

left $\Rightarrow$ close = 1, Right $\Rightarrow$ open = 1

These two don't combine. (opp. direction)

if A node is $\begin{matrix} \text{open} = 2, \\ \text{close} = 1, \\ \text{full} = 0. \end{matrix}$

$\Rightarrow \underbrace{) ((}_{\text{why not}} \text{full.}$

Note: node = $\begin{pmatrix} 1 \\ 1 \\ 5 \end{pmatrix}$

we can one possible string out of this.
open = 1 must have come from Right
and close = 1 from left.

$\rightarrow) () () () () ($

Queries

1. complete overlap $\Rightarrow$ segment (low, high) lies inside the
query = $[l, r]$.

$\rightarrow [l \dots \text{low} \dots \text{high} \dots r]$

$\Rightarrow$ return $\begin{pmatrix} \text{open} \\ \text{close} \\ \text{full} \end{pmatrix}$

2. non overlap.

$[l \dots r] \dots [\text{low} \dots \text{high}]$ or vice-versa.

$\Rightarrow$ return $\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \Rightarrow$ return 0.

$\hookrightarrow$ does not contribute.

3. partial overlap. $\Rightarrow$ main counting and Adding
case.

Left and Right calls will be made.

merge the two Nodes

Merge Nodes Logic for this Question

$$\text{Left} = \begin{pmatrix} \text{open} \\ \text{close} \\ \text{full} \end{pmatrix}$$

$$\text{Right} = \begin{pmatrix} \text{open} \\ \text{close} \\ \text{full} \end{pmatrix}$$

$$\text{new-full} = \min(\text{Left.open}, \text{Right.close})$$

$$\text{full} = \text{Left.full} + \text{Right.full} + \text{new-full}.$$

$$\text{open} = \text{Left.open} - \underbrace{\text{new-full}}_{\substack{\uparrow \\ \text{This many Left.open were used}}} + \text{Right.open}.$$

$$\text{close} = \text{Left.close} + \text{Right.close} - \underbrace{\text{new-full}}_{\substack{\text{This many Right.close were used up.}}}$$

Count Inversions

count no. of pairs such that $(i < j)$ and $a[i] > a[j]$
left > Right.

Ex: arr = {6, 4, 5, 3, 1, 2}

↳ 10 inversion pairs.

⇒ can be solved using Merge Sort (Easily)

Now Solve using Segment Trees.

arr =

6	4	5	3	1	2
---	---	---	---	---	---

↑
iterate over every
element

Take the frequency array.

1	1	1	1	1	1
---	---	---	---	---	---

1 2 3 4 5 6

count how many pairs that can be formed with 6.

(6,)
↑

1	1	1	1	1	1
---	---	---	---	---	---

 → 0.
↔ freq.
1 2 3 4 5 6

All those that are on the left of 6 whose freq > 0.
in freq. map.

⇒ Range Sum of the freq. Map.

↳ Sum(1, 6) = 5.

Now move to next element

count all such (4,)
↑

↳ Sum(1, 4) = 3. freq.

6	4	5	3	1	2
---	---	---	---	---	---

1	1	1	1	1	1
---	---	---	---	---	---

1 2 3 4 5 6
↑ 0 ↑ 0

So to get the Range Sum on the freq. array using Segment Tree.

And note: After iterating each element make its frequency = 0. $\Rightarrow$ point updates in the Tree.

$\Rightarrow$ Segment Tree on freq. Array to get Range Sum with point updates and Query.

On iterating we reduce the freq. $\Rightarrow$ i.e. update Query to make value $\rightarrow$ curr-value - 1 and to get the RangeSum it is Query.

$\rightarrow$ why not use Prefix Sum $\Rightarrow$ This is Dynamic Range Sum where we also make update to frequencies.

BottleNeck $\Rightarrow$ freq. array $\Rightarrow$ element values $\leq 1e5$. would work.